



Trabajo Práctico 1: arVault

[75.73/TB034] Arquitectura del Software
Primer cuatrimestre de 2026

Integrantes:

Apellido y Nombre	Padrón	Mail
Gentilini, Franco	108733	fgentilini@fi.uba.ar
Koo, Hangyeol	108401	hkoo@fi.uba.ar
Sparta, Mateo	109418	msparta@fi.uba.ar

Fecha de entrega: 07/05/2026

Índice

1. Introducción	2
2. Analizando el estado actual	2
2.1. Diagrama Components & Connector	2
2.2. Tests implementados y el por qué.	2
2.3. Análisis de QAs	6
2.3.1. Availability	6
2.3.2. Reliability	6
2.3.3. Performance	7
2.3.4. Scalability	7
2.3.5. Security	7
2.3.6. Visibility	8
2.3.7. Testability	8
2.3.8. Manageability	8
3. Definiendo los objetivos	8
4. Diseñando los cambios e identificando los trade-offs	9
5. Implementación de los cambios	10
5.1. Redis como almacenamiento compartido con operaciones atómicas	10
5.2. Endurecimiento de validaciones y contratos	14
5.3. Balanceo de carga con múltiples instancias	22
5.4. Healthchecker para recuperación automática	24
6. Conclusión	25

1. Introducción

Este trabajo evalúa la arquitectura y el comportamiento operativo del servicio de cambio de monedas de arVault, con foco en atributos de calidad relevantes para un sistema fintech. El objetivo es establecer una línea base técnica y funcional, identificar riesgos arquitectónicos y justificar cambios de diseño con evidencia.

La evaluación combina:

- Análisis arquitectónico del caso base.
- Pruebas funcionales y de known issues.
- Pruebas de performance con escenarios representativos.
- Observabilidad técnica y de negocio mediante métricas.

El resultado esperado es comparar el estado actual con una versión mejorada, discutiendo explícitamente trade-offs entre atributos de calidad.

2. Analizando el estado actual

2.1. Diagrama Components & Connector

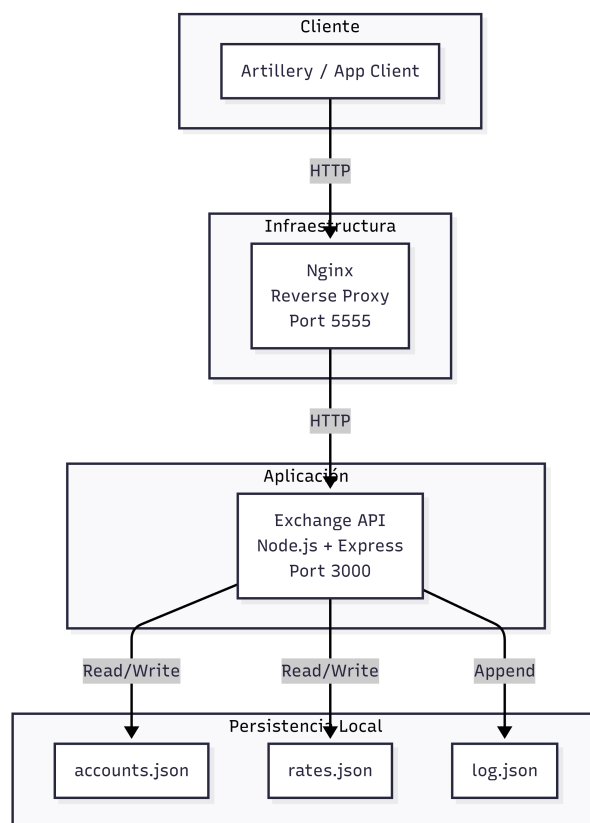


Figura 1: Components & Connector del caso base

2.2. Tests implementados y el por qué.

Se implementaron dos capas complementarias de verificación para instrumentar el estado del caso base y establecer criterios explícitos de mejora.

Tests funcionales

- **Suite de baseline (8 tests):** Congela el comportamiento actual del servicio para detectar regresiones involuntarias durante refactorings. Valida contratos HTTP de lectura, flujos exitosos de exchange con saldos consistentes y casos de error esperado.
- **Suite de known issues (18 tests):** Formaliza los problemas identificados en el análisis del código como criterios de aceptación. Documenta de forma objetiva qué comportamientos son incorrectos (validación débil, errores HTTP ambiguos, falta de atomicidad, etc.).

Escenarios de carga

- **Load test:** Simula carga esperada y creciente en /exchange para establecer el baseline de comportamiento bajo estrés sostenido y detectar degradación. Evalúa Performance, Availability y Reliability.
- **Spike test:** Simula picos abruptos de tráfico (ej: cambios bruscos de dólar) para validar resiliencia del sistema ante cambios inesperados y capacidad de recuperación.
- **Endurance test:** Mantiene carga moderada durante 15 minutos para detectar problemas acumulativos (memory leaks, degradación, acumulación de conexiones). Evalúa Availability, Reliability y Performance prolongada.

Mediciones del caso base

Se incluyen capturas de Grafana para los tres escenarios de carga ejecutados sobre el caso base. En cada escenario se muestran tres paneles complementarios para observar throughput, latencia y comportamiento general del sistema durante la prueba. Estos datos fueron utilizados como referencias para determinar el impacto de los futuros cambios.

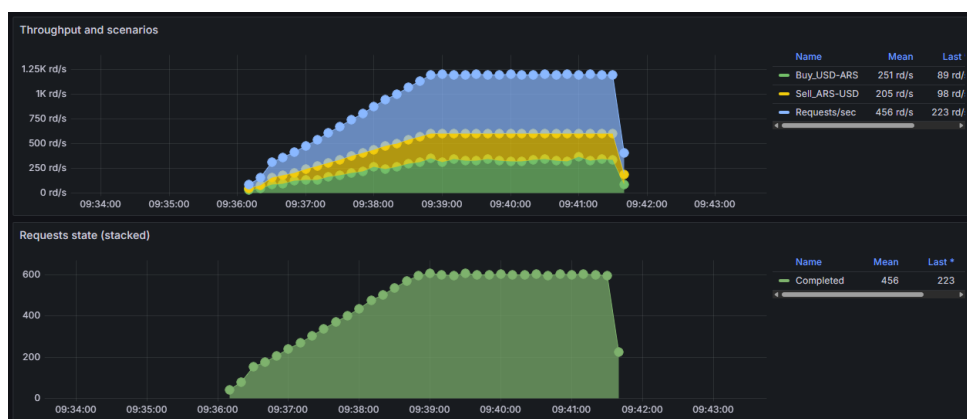


Figura 2: Caso base – load test, Throughput y Requests state

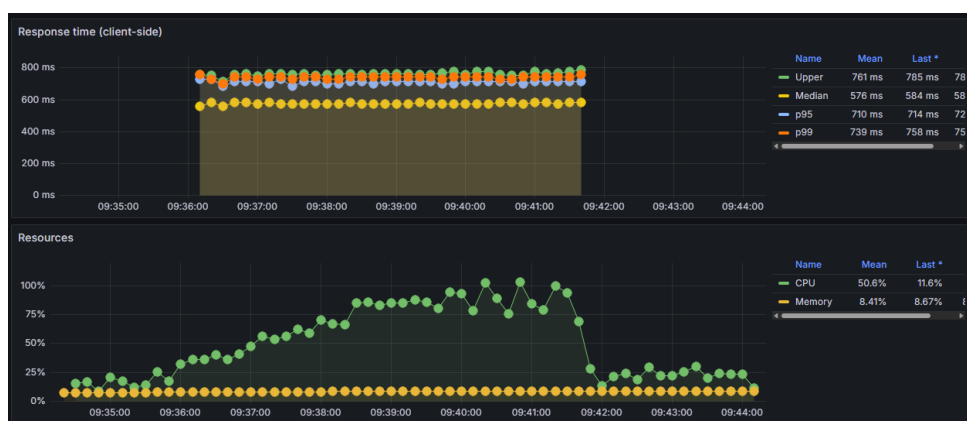


Figura 3: Caso base – load test, Response time y Resources

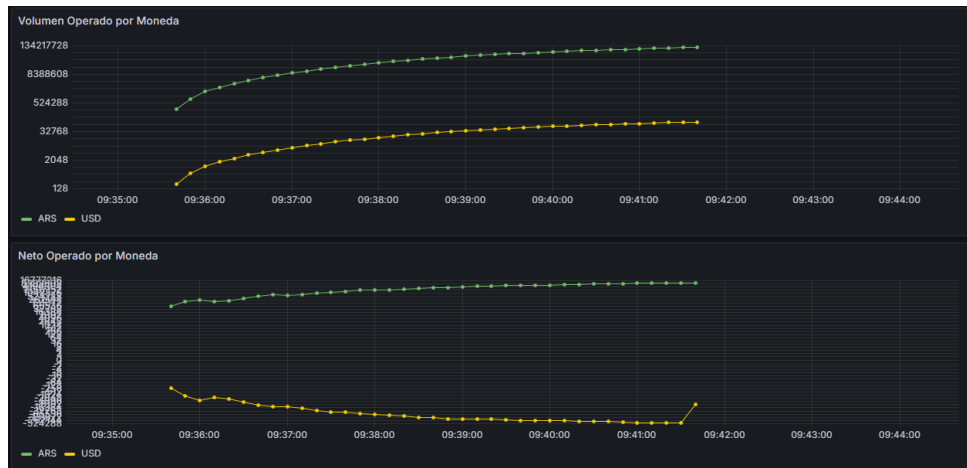


Figura 4: Caso base – load test, Volumen Operado por Moneda y Neto Operado por Moneda

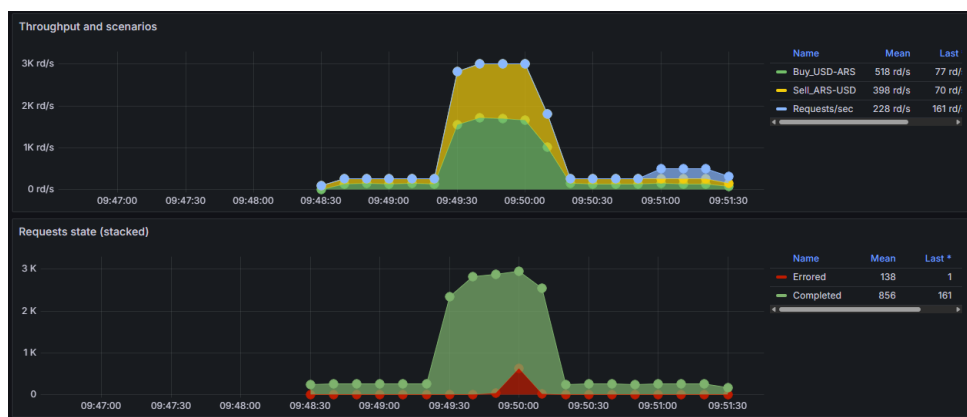


Figura 5: Caso base – spike test, Throughput y Requests state

En la anterior figura es posible observar que el sistema base produce errores frente pico de requests insostenible por la arquitectura del momento. Otro indicio de falta de capacidad es el response time, es evidente que el sistema se encuentra saturado y responde en términos de segundos, con uso de CPU cercano al 500%:

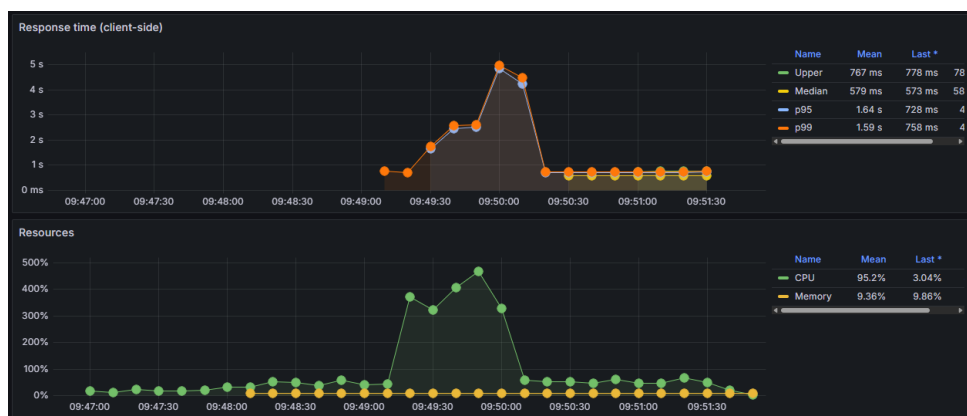


Figura 6: Caso base – spike test, Response time y Resources



Figura 7: Caso base – spike test, Volumen Operado por Moneda y Neto Operado por Moneda

En endurance test, el sistema no parece tener grandes problemas, aunque tiene un crecimiento mínimo en memoria al acumular datos a lo largo del tiempo.

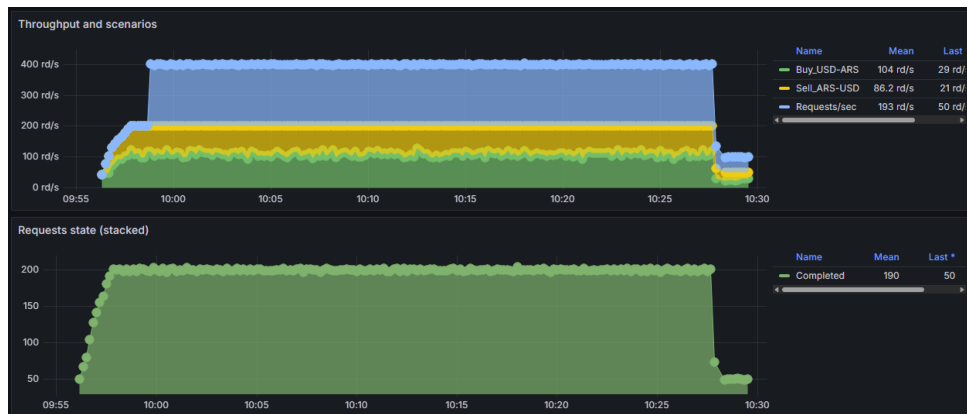


Figura 8: Caso base – endurance test, Throughput y Requests state

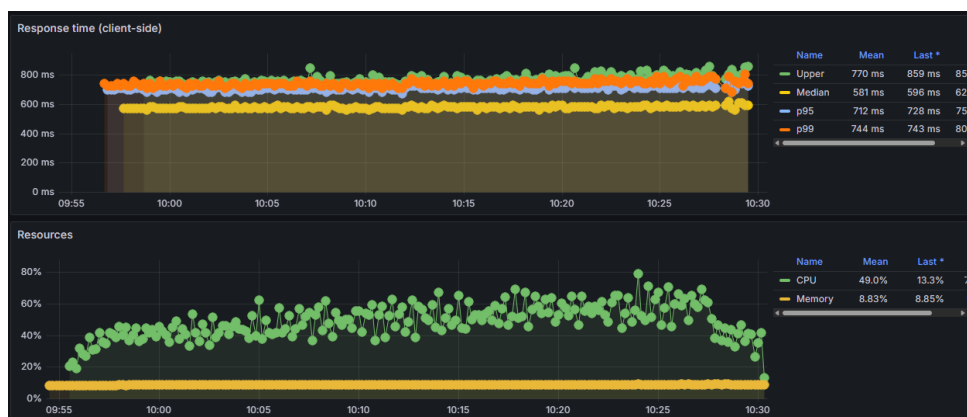


Figura 9: Caso base – endurance test, Response time y Resources

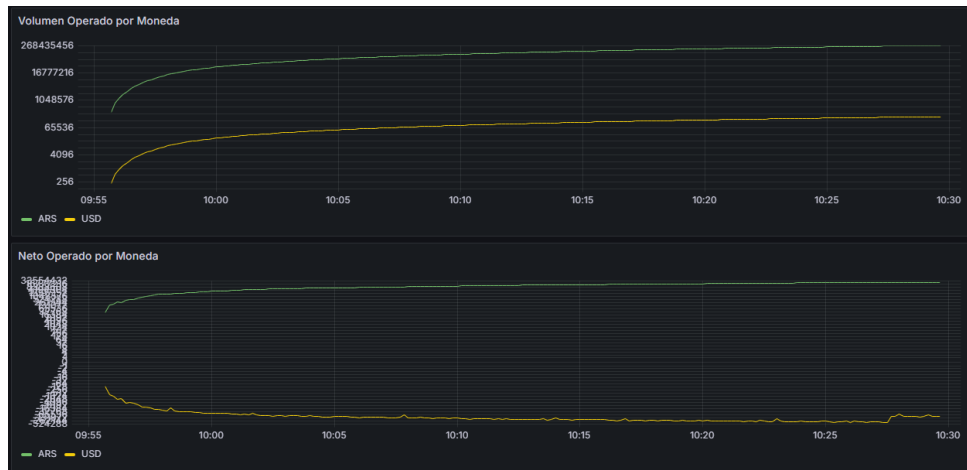


Figura 10: Caso base – endurance test, Volumen Operado por Moneda y Neto Operado por Moneda

2.3. Análisis de QAs

2.3.1. Availability

El sistema muestra una disponibilidad aceptable solo en condiciones simples de uso, pero su diseño actual lo vuelve frágil frente a fallas de ejecución y ante crecimiento de tráfico. El riesgo principal no es solo una caída total, sino también la degradación del servicio por errores de validación, inconsistencias de estado o escrituras parciales en disco.

Problemas que afectan este atributo:

- No hay un mecanismo claro de degradación controlada.
- Algunas operaciones pueden terminar en errores no esperables para el cliente.
- El estado persiste en archivos locales, por lo que una falla del proceso puede dejar el sistema indisponible hasta restaurar los datos.
- Si el guardado se interrumpe a mitad de camino, el sistema puede quedar en un estado corrupto.

En este dominio, la disponibilidad no se limita a que el proceso siga vivo: también implica que el servicio responda correctamente y preserve coherencia operativa.

2.3.2. Reliability

Este es uno de los atributos más comprometidos del caso base. El servicio tiene varios puntos donde una operación válida o inválida no se maneja de forma robusta, lo que hace que el comportamiento sea difícil de predecir y riesgoso para un entorno financiero.

Hallazgos relevantes:

- La app parece estar pensada como si todos los usuarios compartieran una misma cuenta por moneda, en lugar de modelar aislamiento real entre usuarios.
- No hay validación suficiente de los campos de entrada, en muchos casos el código solo verifica que existan, pero no que tengan formato o semántica válidos.
- Hay falta de atomicidad en múltiples flujos críticos, especialmente en exchange y en el guardado del estado.
- Existen rutas con contratos poco claros o directamente raros, lo que dificulta confiar en la respuesta del sistema.
- El endpoint de log expone todo el historial acumulado, sin paginación ni límites, lo que además de ser un problema de performance también afecta confiabilidad operativa.

- La función de transferencia tiene valores hardcodedos y un delay aleatorio, lo que introduce variabilidad artificial en el comportamiento.

En términos de software financiero, esto significa que el sistema no garantiza consistencia ni predictibilidad, dos condiciones básicas para ser confiable.

2.3.3. Performance

En lectura simple el sistema puede responder bien, pero el diseño actual limita el comportamiento cuando se ejecutan operaciones reales de exchange. El mayor costo no está en el cálculo en sí, sino en la forma en que se maneja el estado y en la ausencia de validaciones tempranas.

Factores que impactan performance:

- Se cargan en memoria accounts, rates y log completos.
- El log se reescribe completo en vez de anexarse.
- Se persiste estado aunque no haya cambios.
- Se realizan escrituras periódicas por constantes hardcodedas, incluso cuando podrían coincidir con una transacción en curso.
- El endpoint de log puede crecer indefinidamente y volverse cada vez más costoso.

Esto hace que la performance no sea solo una cuestión de latencia puntual, sino también de escalabilidad del costo operativo a medida que el sistema acumula datos.

2.3.4. Scalability

La escalabilidad horizontal está bloqueada por el diseño de almacenamiento actual. Como el estado vive enteramente en memoria y se sincroniza con archivos locales, no hay una fuente compartida de verdad que permita levantar múltiples instancias sin inconsistencia.

Problemas clave:

- Los datos viven separados por archivo y se mantienen en memoria.
- No hay un storage externo compartido.
- La escritura no es atómica.
- El persistido periódico y los guardados completos vuelven muy difícil repartir carga entre instancias.

Mientras el sistema siga así, escalar horizontalmente no es viable sin riesgo de pérdida de consistencia.

2.3.5. Security

Aunque el enunciado asume que la autenticación/autorización se resuelve fuera de este servicio, el código actual no refuerza ese supuesto con defensas mínimas.

Problemas observados:

- No hay middleware de seguridad en la API.
- Los endpoints aceptan demasiadas combinaciones inválidas de entrada.
- No se valida con suficiente rigor el formato ni la intención de los requests.

Esto no reemplaza vaultSec, pero sí deja claro que el servicio no está preparado para defenderse frente a entradas malformadas o maliciosas por sí mismo. En otras palabras, depende demasiado de la capa externa.

2.3.6. Visibility

La visibilidad del sistema en el caso base es baja, porque no expone suficiente información para entender con claridad qué ocurre internamente durante una operación de exchange. Eso dificulta detectar cuellos de botella, validar hipótesis y correlacionar fallos con cambios de comportamiento.

Problemas que afectan este atributo:

- No hay suficientes métricas para observar el comportamiento interno del servicio.
- El estado de negocio y el estado técnico no se pueden leer de forma simple y centralizada.
- El endpoint de log expone información, pero no de manera pensada para monitoreo ni análisis operacional.
- No hay un mecanismo claro para seguir la evolución de la carga, la latencia o el impacto de negocio en tiempo real.

2.3.7. Testability

La testabilidad del sistema es razonable como punto de partida: los endpoints son accesibles, el estado es observable y no hay lógica oculta en componentes separados. Esto significa que es posible verificar el comportamiento del sistema de forma directa.

Lo positivo:

- Se pueden probar endpoints directamente sin necesidad de herramientas complejas.
- El estado es visible y observable.
- Los flujos son secuenciales y predecibles en el camino de éxito.

Lo negativo:

- La validación insuficiente hace difícil definir contratos estables que permanezcan a través de refactors.
- La falta de atomicidad complica las pruebas reproducibles, especialmente bajo concurrencia.
- El comportamiento interno depende demasiado de la forma en que se guardan datos y del orden de ejecución.

2.3.8. Manageability

La manageability del caso base es baja, principalmente por el uso de almacenamiento local en archivos JSON, la persistencia periódica con lógica fija y la falta de mecanismos claros de recuperación ante fallas. Operar este sistema en producción es complejo porque el estado puede quedar inconsistente, las escrituras no son atómicas y no hay una estrategia sólida para escalar o recuperar el servicio de forma confiable.

Problemas que afectan este atributo:

- El estado se guarda en archivos locales en lugar de en un storage compartido y administrable.
- La persistencia se realiza con lógica fija y periódica, lo que complica el control operativo.
- No hay una estrategia clara de recuperación ante fallas parciales (ej: si se interrumpe el save durante una escritura).
- La dependencia de archivos locales hace imposible operar múltiples instancias del servicio sin sincronización manual.

3. Definiendo los objetivos

Para fundamentar la estrategia de mejora, identificamos los atributos de calidad prioritarios en un sistema de cambio de monedas, considerando los fundamentos del rubro financiero y el estado actual del caso base.

Atributos primarios (críticos para el dominio):

- **Availability y Reliability:** En un sistema financiero, disponibilidad y consistencia son no negociables. Fallos operacionales generan pérdida de fondos, reputación dañada y pérdida de usuarios. El análisis del código base muestra problemas graves en ambos atributos que requieren corrección inmediata.
- **Performance:** La sensibilidad del usuario a la latencia impacta directamente en experiencia y decisión de compra. Un sistema lento pierde operaciones.

Atributos secundarios (necesarios para ejecutar cambios):

- **Visibility, Manageability y Testability:** Son necesarios como herramientas para instrumentar cambios, medir impacto real, detectar regresiones y reducir riesgo operativo durante refactors.

Aspectos fuera del scope:

- **Security:** Autenticación y autorización están delegadas a vaultSec, un servicio externo dedicado, por lo que no forman parte del análisis en este trabajo.

Esta priorización guía tanto la selección de tácticas como el criterio de éxito en el informe: los cambios propuestos se evaluarán principalmente por su impacto en Availability, Reliability y Performance, con visibilidad clara mediante observabilidad.

4. Diseñando los cambios e identificando los trade-offs

Con base en el análisis del caso base, se definieron las siguientes tácticas de mejora:

1. Redis como almacenamiento compartido

La migración del estado desde archivos locales hacia Redis permite centralizar la persistencia, reducir inconsistencias entre instancias y habilitar escalabilidad horizontal.

- Trade-off: mayor complejidad de infraestructura.
- Beneficio principal: mejora de reliability y scalability.

2. Endurecimiento de validaciones y contratos

La incorporación de validaciones más estrictas y documentación más precisa de los endpoints reduce comportamientos inesperados y mejora la robustez del servicio.

- Trade-off: aumento de complejidad en la lógica de negocio.
- Beneficio principal: mejora de reliability.

3. Balanceo de carga con múltiples instancias

La distribución del tráfico entre varias instancias del servicio, apoyada en un almacenamiento compartido, permite absorber mejor picos de demanda y reducir el impacto de fallas puntuales.

- Trade-off: mayor complejidad operativa.
- Beneficio principal: mejora de availability, reliability y scalability.

4. Healthchecker para recuperación automática

Se incorporó un healthchecker que monitorea periódicamente las instancias de API y nginx mediante el endpoint /healthcheck. Ante fallas consecutivas reinicia el contenedor que se determinó caído.

- Trade-off: degradación de simplicidad.
- Beneficio principal: mejora de availability, reliability y manageability al reducir esfuerzo manual ante fallas.

5. Implementación de los cambios

5.1. Redis como almacenamiento compartido con operaciones atómicas

Se reemplazó el esquema de persistencia basado en archivos locales por un almacenamiento compartido en Redis para cuentas, tasas y log de operaciones. Este cambio apunta a eliminar inconsistencias de estado entre procesos, reducir el riesgo de corrupción por escrituras parciales y preparar la arquitectura para escalar horizontalmente.

Diagrama Components & Connector

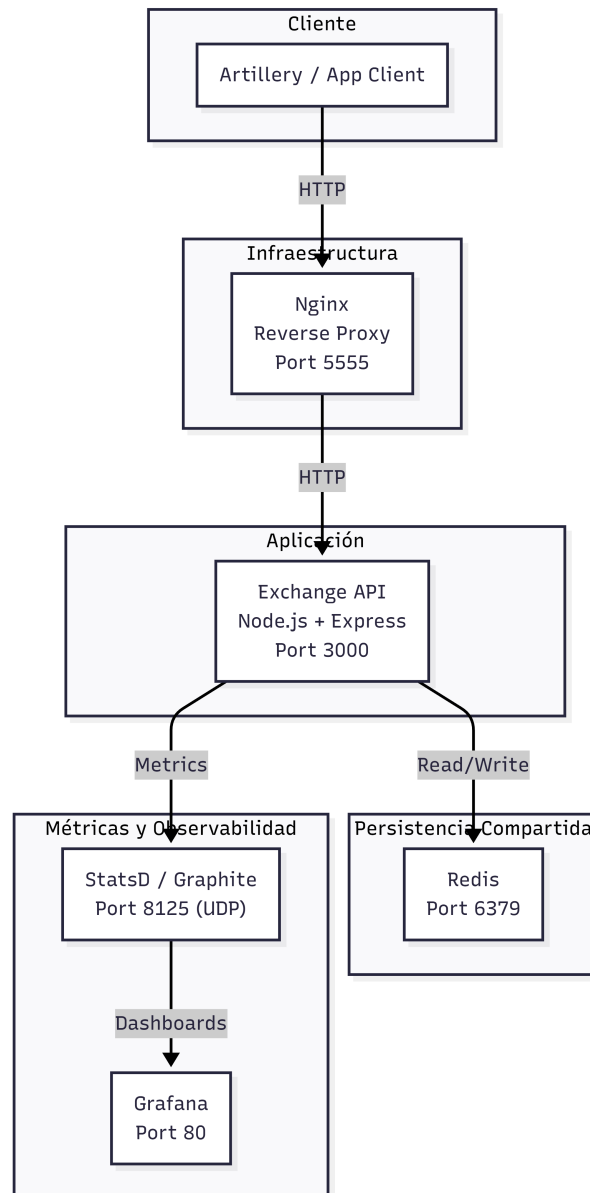


Figura 11: Components & Connector Redis

Cambios de QAs justificados

- **Reliability (mejora):** el estado deja de depender de archivos locales y pasa a una fuente compartida, reduciendo inconsistencias y errores por persistencia parcial.

- **Availability (mejora)**: al desacoplar cómputo y persistencia, una instancia puede reiniciarse sin perder estado local, disminuyendo el impacto operativo de fallas puntuales.
- **Scalability (mejora)**: múltiples instancias del servicio pueden operar sobre el mismo estado, habilitando escalado horizontal real.
- **Performance (mejora parcial)**: se evitan reescrituras completas de archivos y se aprovechan operaciones eficientes en memoria, el costo es un salto de red adicional hacia Redis.
- **Manageability (mejora)**: centralizar datos simplifica operación, inspección y administración del estado del sistema.
- **Visibility (mejora indirecta)**: al tener un backend de estado único, resulta más simple correlacionar métricas y comportamiento real del sistema.
- **Simplicity (degrada)**: aumenta la complejidad de infraestructura y operación (nuevo componente, configuración, dependencia de red).

Mediciones y pruebas

Las mediciones de esta variante muestran el comportamiento del sistema una vez centralizado el estado en Redis. Se mantienen los mismos tres escenarios de carga para poder comparar la evolución respecto del caso base.

Como es de esperar, la mayoría de las métricas no se ven afectadas por este cambio, aunque sí hubo un crecimiento en uso de recursos: cercano a un 30 % en CPU, y un 10 % en memoria. Esto provoca que en momentos de saturación en spike test, el performance se vea degradado aún peor que el caso base, con aumento en cantidad de errores (de 138 a más de mil) y en response time (de 5 a 8 segundos) durante el pico.

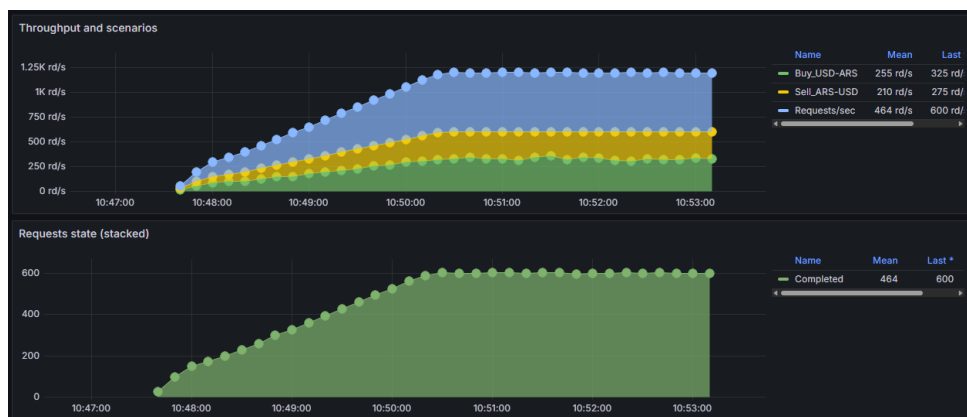


Figura 12: Redis – load test, Throughput y Requests state

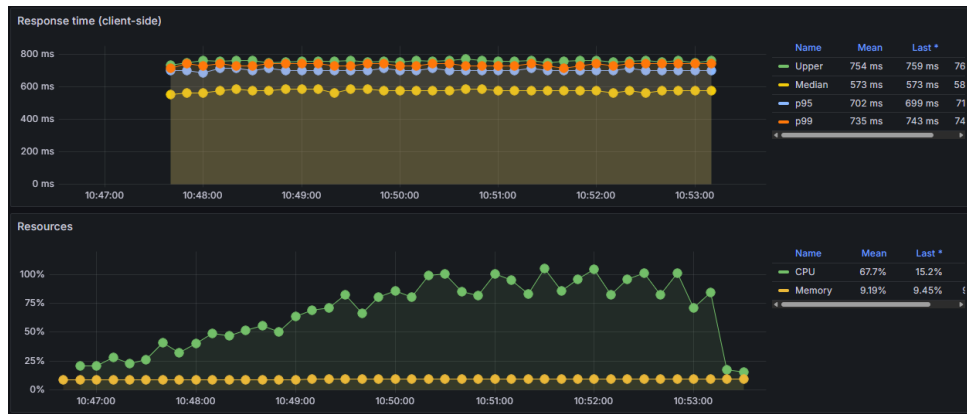


Figura 13: Redis – load test, Response time y Resources

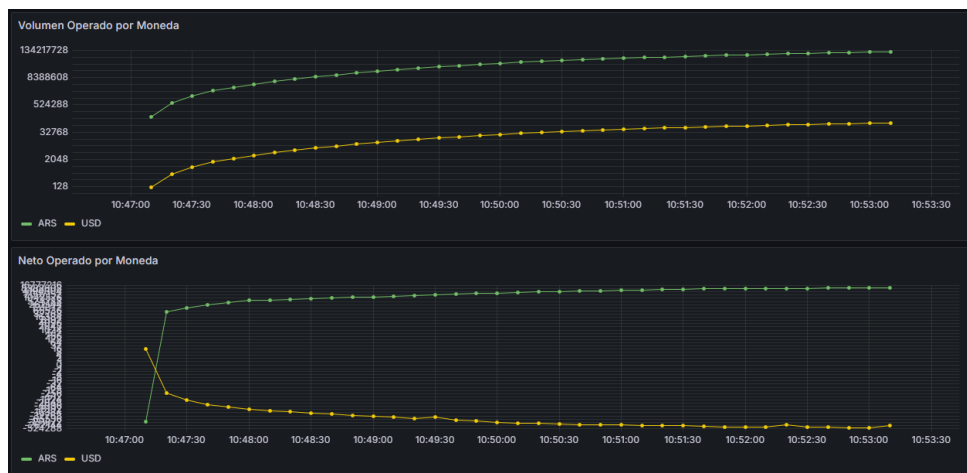


Figura 14: Redis – load test, Volumen Operado por Moneda y Neto Operado por Moneda

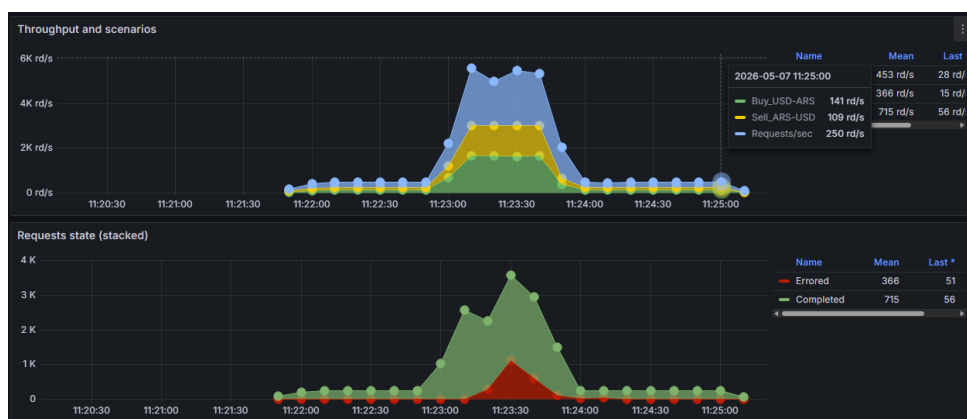


Figura 15: Redis – spike test, Throughput y Requests state

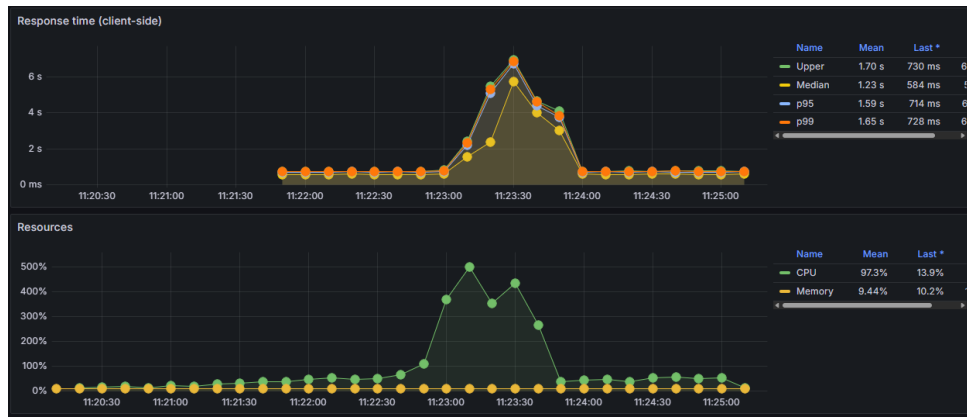


Figura 16: Redis – spike test, Response time y Resources



Figura 17: Redis – spike test, Volumen Operado por Moneda y Neto Operado por Moneda

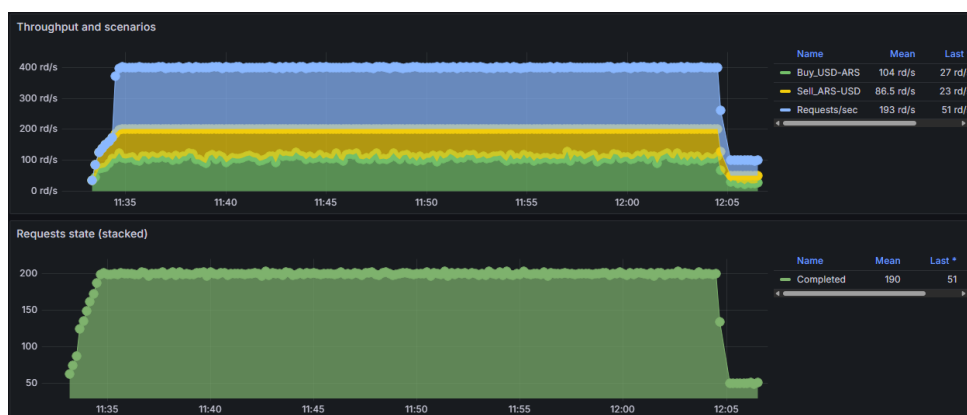


Figura 18: Redis – endurance test, Throughput y Requests state



Figura 19: Redis – endurance test, Response time y Resources

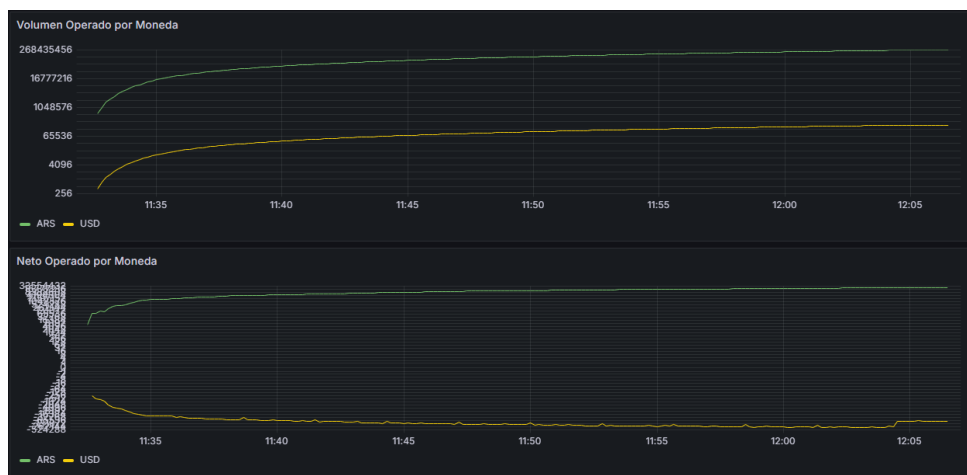


Figura 20: Redis – endurance test, Volumen Operado por Moneda y Neto Operado por Moneda

5.2. Endurecimiento de validaciones y contratos

Esta mejora se orienta a reforzar la capa de entrada del servicio mediante validaciones estrictas de parámetros, normalización de contratos HTTP y respuestas de error consistentes. El objetivo es reducir comportamientos ambiguos, evitar fallas por entradas inválidas y hacer más predecible el comportamiento del sistema bajo condiciones no ideales.

Diagrama Components & Connector

En esta mejora no se modifica la topología de componentes, sino la lógica de validación y contrato en la API. Por lo tanto, el C&C se mantiene equivalente al caso con Redis.

Cambios de QAs justificados

- **Reliability (mejora):** Se incorporan validaciones de tipo, rango y formato para entradas críticas (montos, IDs, monedas, tasas), evitando que requests inválidos deriven en comportamiento inconsistente. Además, se tipifican mejor errores funcionales (por ejemplo, conflictos de negocio frente a errores de cliente).
- **Availability (mejora indirecta):** Al filtrar tempranamente requests inválidos, se evita consumo innecesario de recursos y se reduce la probabilidad de fallas evitables en tiempo de ejecución.

- **Performance (mejora leve):** La validación temprana evita recorrer lógica de negocio y persistencia cuando la request es inválida. El costo de validación es bajo frente al costo evitado de procesamiento posterior.
- **Manageability (mejora):** Contratos y códigos de error más consistentes facilitan operación, diagnóstico y soporte, porque el comportamiento ante entradas erróneas pasa a ser explícito y repetible.
- **Testability (mejora):** Al definir respuestas determinísticas para casos inválidos (400/404/409 según corresponda), el sistema se vuelve más verificable y con menor ambigüedad al validar regresiones.
- **Visibility (mejora indirecta):** La estandarización de errores permite interpretar mejor métricas de fallas y separar errores de cliente de incidentes reales de infraestructura o disponibilidad.
- **Simplicity (degrada):** Disminuye la simplicidad de la capa de entrada (más reglas y más ramas de control), pero se obtiene un aumento claro de robustez y previsibilidad del servicio.

Mediciones y pruebas

Para esta variante se incluyen mediciones de tres configuraciones de despliegue, de menor a mayor capacidad: una instancia de Nginx y una instancia de API, una instancia de Nginx y tres instancias de API, y dos instancias de Nginx con tres instancias de API. Esto permite mostrar cómo evoluciona el sistema a medida que se escala la capa de entrada y de procesamiento.

Configuración con 1 instancia de Nginx y 1 instancia de API: Resulta lógico que esta configuración no muestre cambios significativos, aunque debido al código adicional para soportar múltiples instancias sí hubo un pequeño crecimiento en el uso de recursos: un 5 % en CPU y un 10 % en memoria.

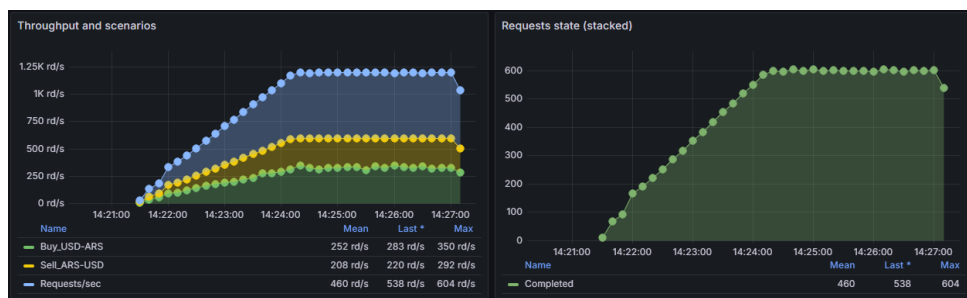


Figura 21: 1 instancia de Nginx y 1 instancia de API – load test, Throughput y Requests state

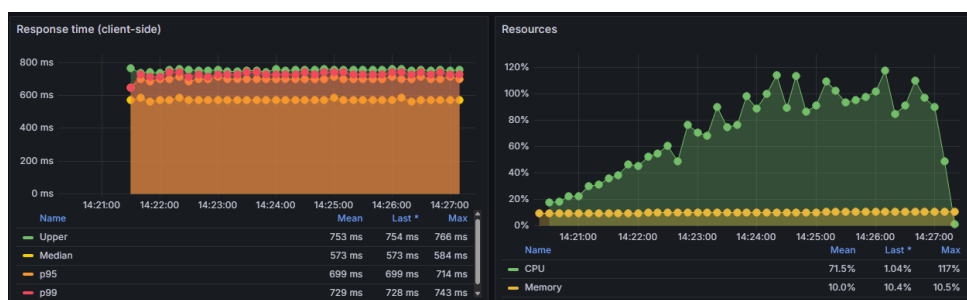


Figura 22: 1 instancia de Nginx y 1 instancia de API – load test, Response time y Resources

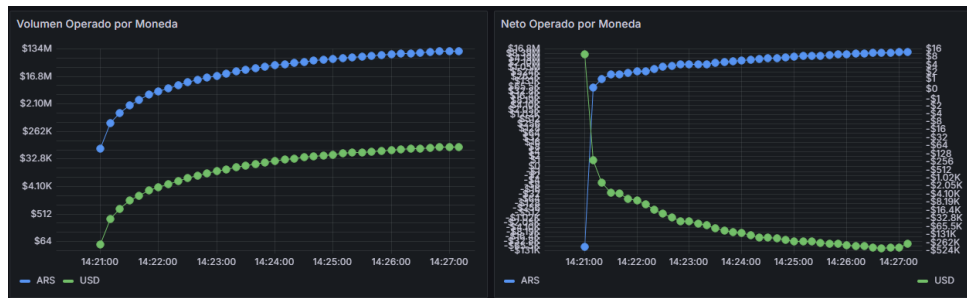


Figura 23: 1 instancia de Nginx y 1 instancia de API – load test, Volumen Operado por Moneda y Neto Operado por Moneda

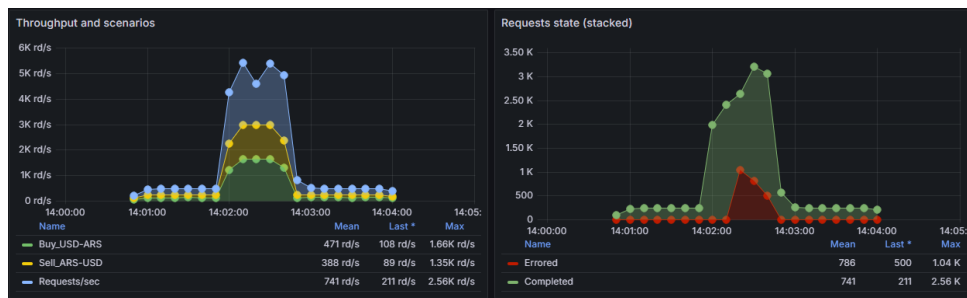


Figura 24: 1 instancia de Nginx y 1 instancia de API – spike test, Throughput y Requests state



Figura 25: 1 instancia de Nginx y 1 instancia de API – spike test, Response time y Resources

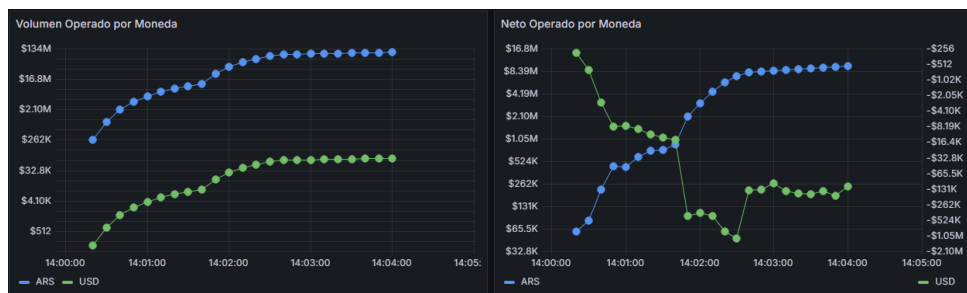


Figura 26: 1 instancia de Nginx y 1 instancia de API – spike test, Volumen Operado por Moneda y Neto Operado por Moneda

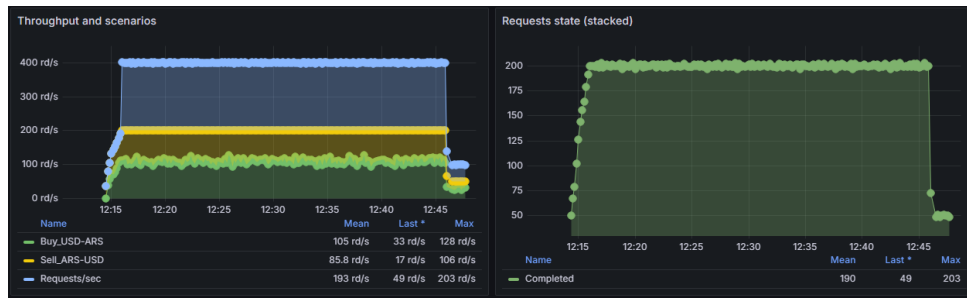


Figura 27: 1 instancia de Nginx y 1 instancia de API – endurance test, Throughput y Requests state

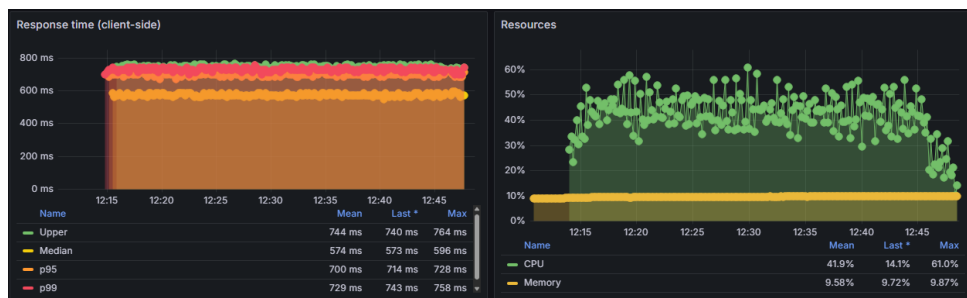


Figura 28: 1 instancia de Nginx y 1 instancia de API – endurance test, Response time y Resources

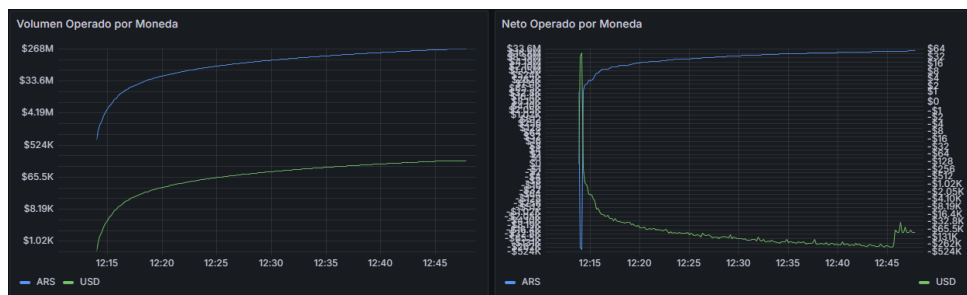


Figura 29: 1 instancia de Nginx y 1 instancia de API – endurance test, Volumen Operado por Moneda y Neto Operado por Moneda

Configuración con 1 instancia de Nginx y 3 instancias de API: Con esta configuración es posible observar que durante el spike test se disminuyó significativamente la cantidad de errores (de más de 2 mil errores en total a 631), aunque siguen presente, y se achicaron los momentos de performance degradada por saturación, lo cual es visible en el gráfico de response time.

Al incorporar más instancias, es evidente el aumento en uso de recursos para todos los casos.

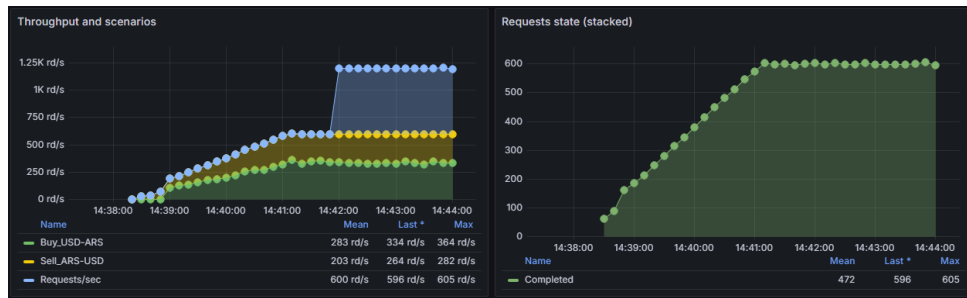


Figura 30: 1 instancia de Nginx y 3 instancias de API – load test, Throughput y Requests state

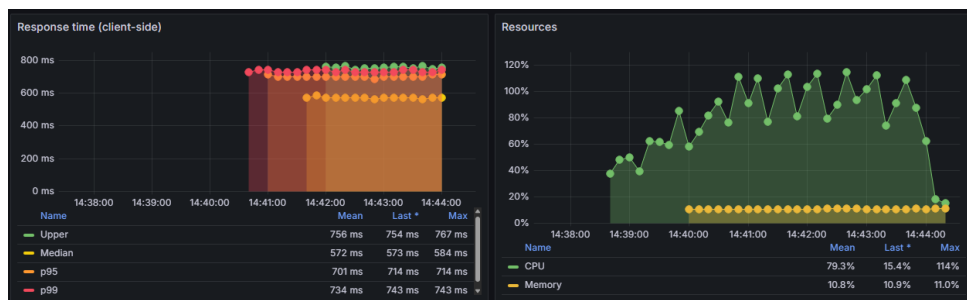


Figura 31: 1 instancia de Nginx y 3 instancias de API – load test, Response time y Resources

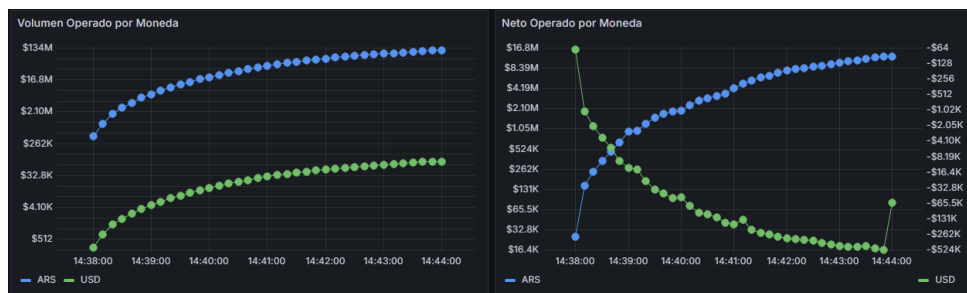


Figura 32: 1 instancia de Nginx y 3 instancias de API – load test, Volumen Operado por Moneda y Neto Operado por Moneda

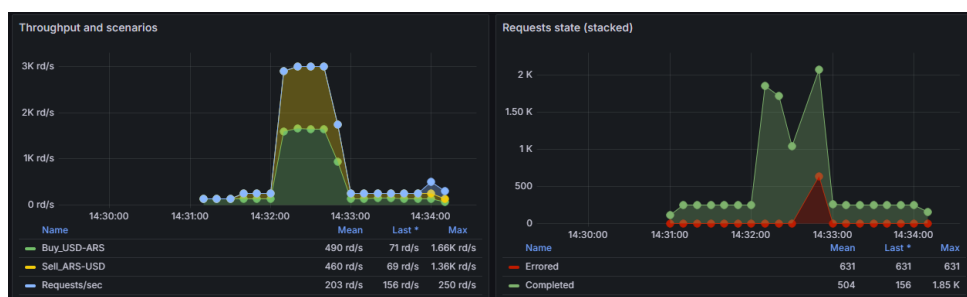


Figura 33: 1 instancia de Nginx y 3 instancias de API – spike test, Throughput y Requests state

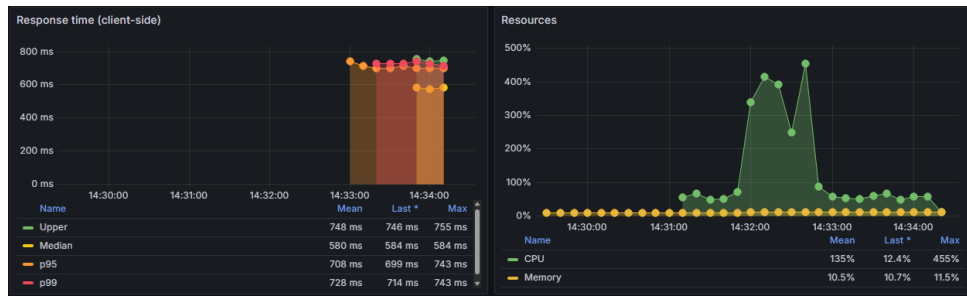


Figura 34: 1 instancia de Nginx y 3 instancias de API – spike test, Response time y Resources

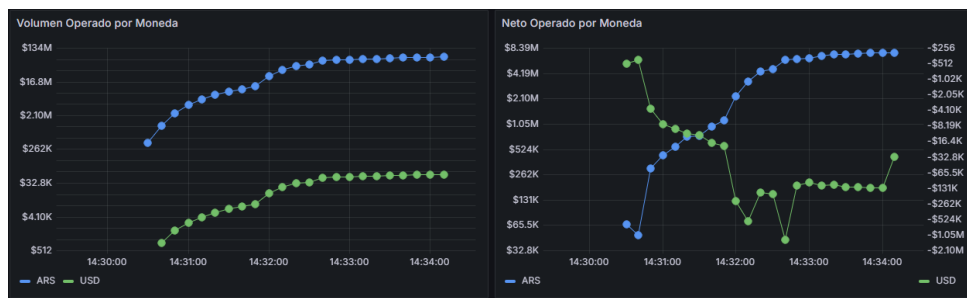


Figura 35: 1 instancia de Nginx y 3 instancias de API – spike test, Volumen Operado por Moneda y Neto Operado por Moneda

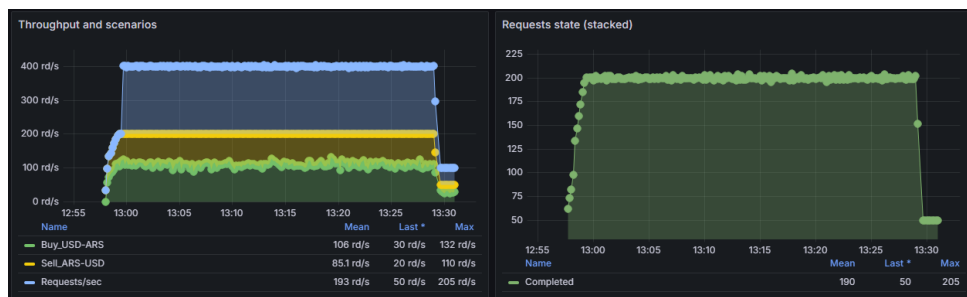


Figura 36: 1 instancia de Nginx y 3 instancias de API – endurance test, Throughput y Requests state

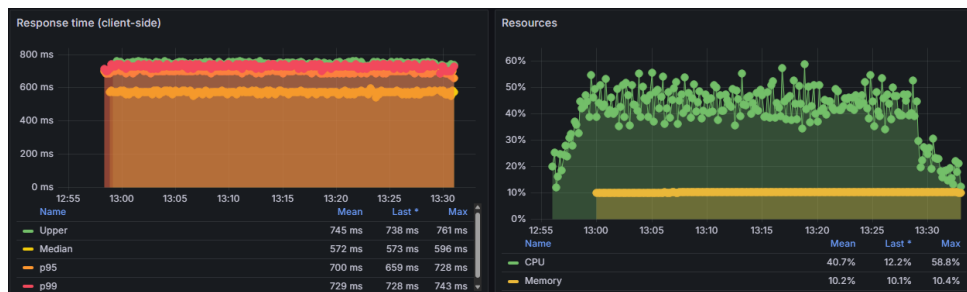


Figura 37: 1 instancia de Nginx y 3 instancias de API – endurance test, Response time y Resources

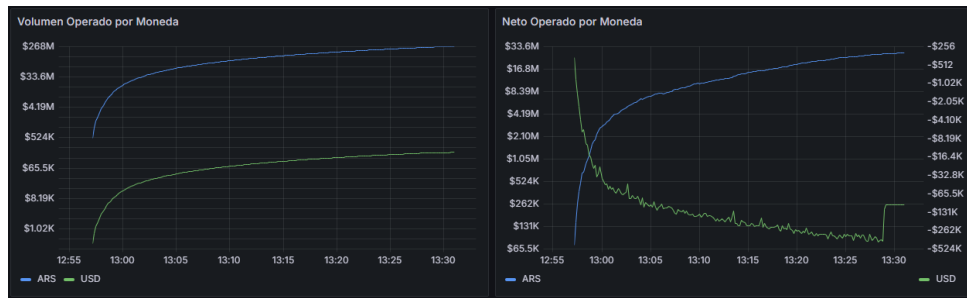


Figura 38: 1 instancia de Nginx y 3 instancias de API – endurance test, Volumen Operado por Moneda y Neto Operado por Moneda

Configuración con 2 instancias de Nginx y 3 instancias de API: Con introducción de instancia adicional de Nginx a la configuración anterior, logramos cubrir casi por completo el caso de pico: resultando en cantidad de errores total de solo 27. Si bien aumentó respecto la configuración anterior el response time en momento de saturación, es posible ver que no pasa los 2 segundos, un valor tolerable a diferencia de 8 segundos con 1 instancia de Nginx y 1 de API.

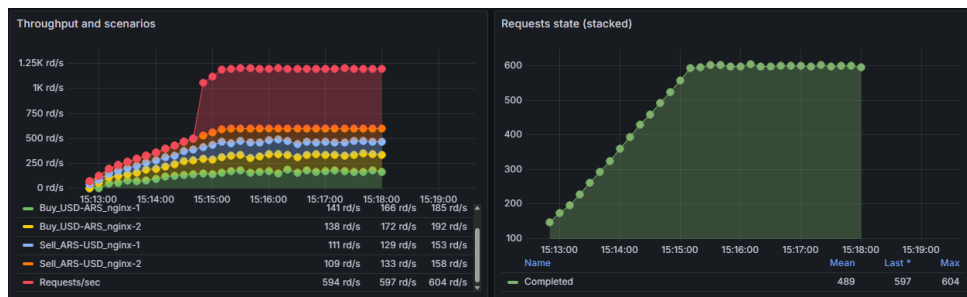


Figura 39: 2 instancias de Nginx y 3 instancias de API – load test, Throughput y Requests state

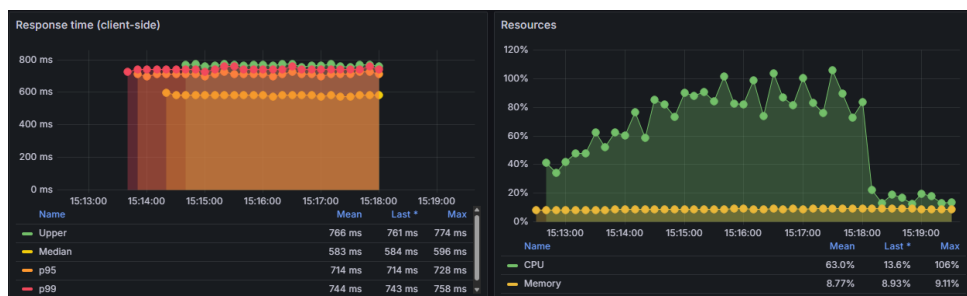


Figura 40: 2 instancias de Nginx y 3 instancias de API – load test, Response time y Resources

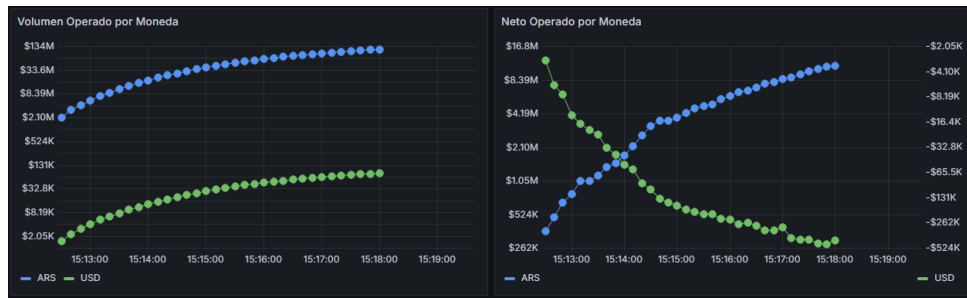


Figura 41: 2 instancias de Nginx y 3 instancias de API – load test, Volumen Operado por Moneda y Neto Operado por Moneda

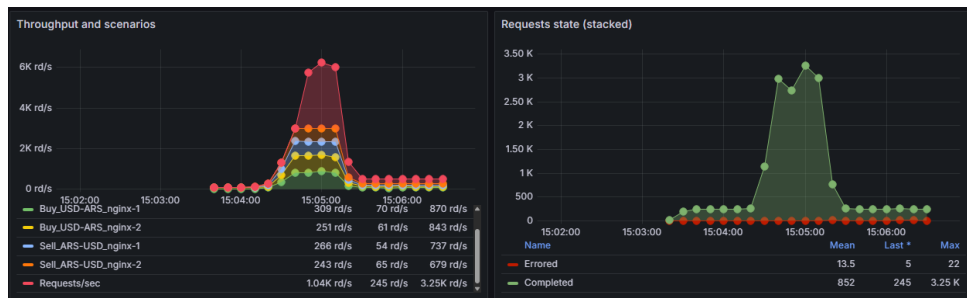


Figura 42: 2 instancias de Nginx y 3 instancias de API – spike test, Throughput y Requests state

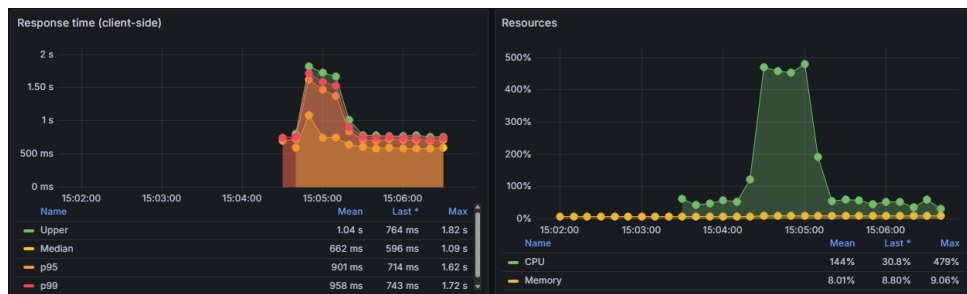


Figura 43: 2 instancias de Nginx y 3 instancias de API – spike test, Response time y Resources

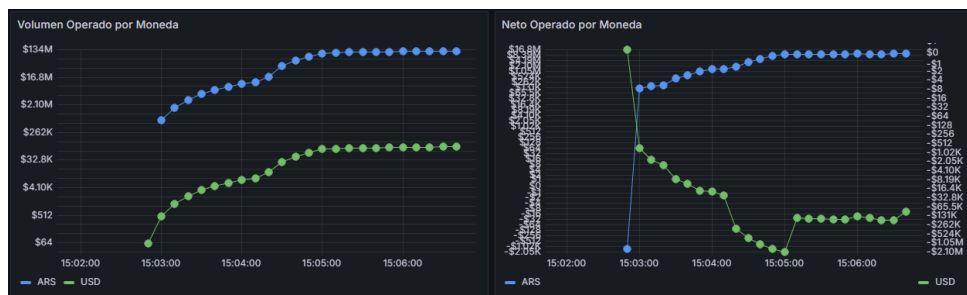


Figura 44: 2 instancias de Nginx y 3 instancias de API – spike test, Volumen Operado por Moneda y Neto Operado por Moneda

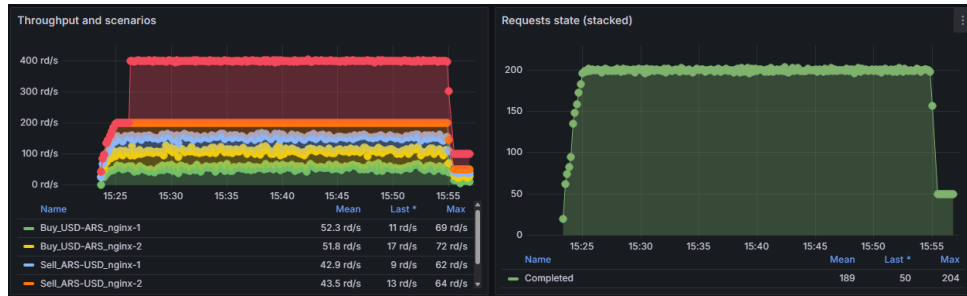


Figura 45: 2 instancias de Nginx y 3 instancias de API – endurance test, Throughput y Requests state

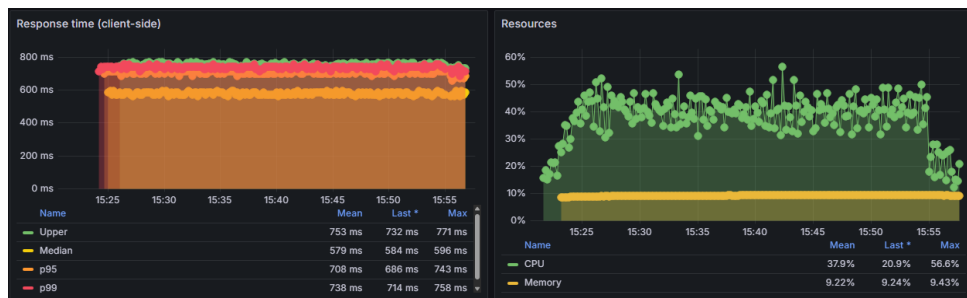


Figura 46: 2 instancias de Nginx y 3 instancias de API – endurance test, Response time y Resources

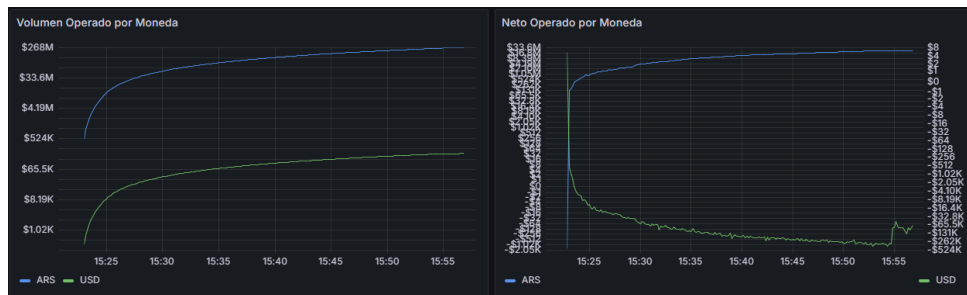


Figura 47: 2 instancias de Nginx y 3 instancias de API – endurance test, Volumen Operado por Moneda y Neto Operado por Moneda

Al ser un cambio más orientado a nivel funcional que casi no afecta la performance, no incluimos mediciones en el informe al no haber cambios.

5.3. Balanceo de carga con múltiples instancias

Esta variante extiende la idea de escalado horizontal a toda la capa de entrada: ya no existe un único Nginx ni una única instancia de la API, sino un conjunto de reverse proxies y réplicas del servicio de negocio distribuidos en puertos distintos. El estado sigue centralizado en Redis, por lo que todas las instancias observan la misma información de cuentas, tasas y log. La consecuencia principal es que el sistema puede absorber mejor picos de demanda y aislar fallas parciales sin detener por completo el servicio.

La configuración de despliegue refleja esa decisión: el archivo `docker-compose-scaled.yml` define `nginx-1`, `nginx-2` y `nginx-3`, todos montando la misma configuración de reverse proxy, mientras que la API se levanta con varias réplicas detrás del nombre de servicio `api`. Para las pruebas de carga, los escenarios de Artillery también se duplican por puerto, de manera que cada instancia de Nginx reciba tráfico de forma explícita y comparable.

Diagrama Components & Connector

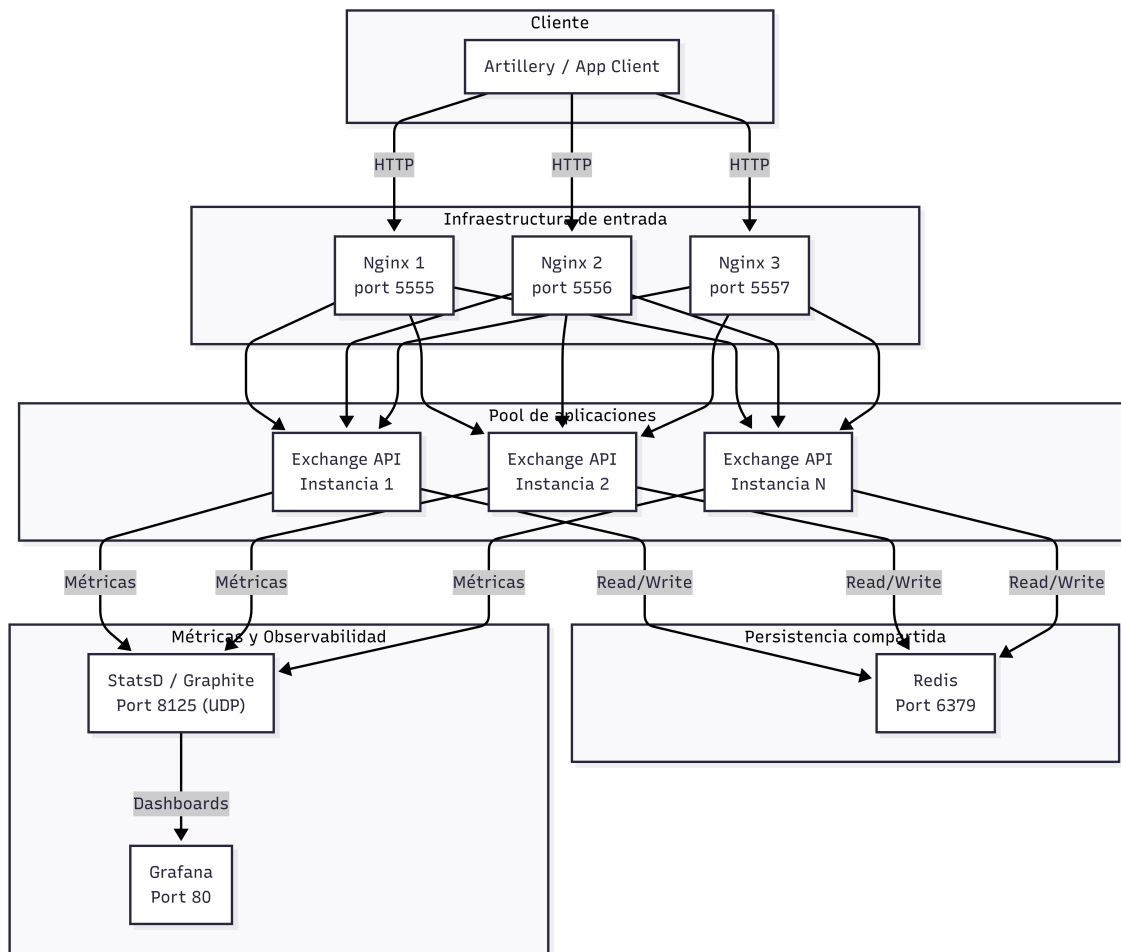


Figura 48: Components & Connector Nginx y API escalados horizontalmente

Cambios de QAs justificados

- **Availability (mejora significativa):** Al tener múltiples Nginx y múltiples réplicas de la API, una falla puntual deja de ser un punto único de caída. El sistema conserva capacidad parcial de atención mientras permanezcan activos otros contenedores del pool.
- **Reliability (mejora):** La carga se reparte entre más procesos, reduciendo la probabilidad de saturación y de comportamientos erráticos por sobreuso de CPU, memoria o concurrencia. Redis centraliza el estado y evita inconsistencias entre réplicas.
- **Performance (mejora en throughput):** El sistema puede sostener más requests simultáneos porque la entrada y el procesamiento se distribuyen. La latencia individual puede mantenerse similar, pero el throughput agregado aumenta.
- **Scalability (mejora significativa):** Esta es la mejora central de la variante. Se puede aumentar la capacidad total agregando instancias de Nginx y de la API sin modificar la lógica de negocio, manteniendo la misma interfaz externa.
- **Visibility (mejora):** El dashboard incorpora un filtro de contenedores que permite distinguir instancias como `exchange-nginx-*` y `exchange-api-*`. Esto facilita comparar el comportamiento de cada réplica y detectar diferencias operativas entre instancias.
- **Manageability (mixta):** El despliegue se vuelve más flexible, porque es posible ampliar o reducir el pool con Compose, pero también crece la complejidad operativa al administrar más contenedores, puertos y rutas de monitoreo.

- **Simplicity (degrada):** La arquitectura es más compleja que en el caso base, ya que introduce balanceo distribuido, más unidades desplegables y más puntos de observación. Ese costo es aceptable porque habilita las mejoras de disponibilidad y escalabilidad.

Mediciones y pruebas

5.4. Healthchecker para recuperación automática

Se implementó un Healthchecker que mediante polling monitorea el estado de las instancias junto al endpoint /healthcheck dedicado para este cambio. Al detectar fallas consecutivas de un contenedor de nginx o API, éste se encarga de reiniciarlo automáticamente utilizando el socket de Docker, garantizando a cierto grado la disponibilidad. Como las instancias de nginx pueden responder con error ante fallas de API, se chequean primero los contenedores de API.

Una posible mejora para extender esto podría ser implementar múltiples instancias de Healthchecker que se monitoreen entre sí para garantizar aún mayor disponibilidad; sin embargo, dado que una sola instancia de healthchecker no implica un único punto de fallo se decidió dejarlo como una futura implementación opcional.

Diagrama Components & Connector

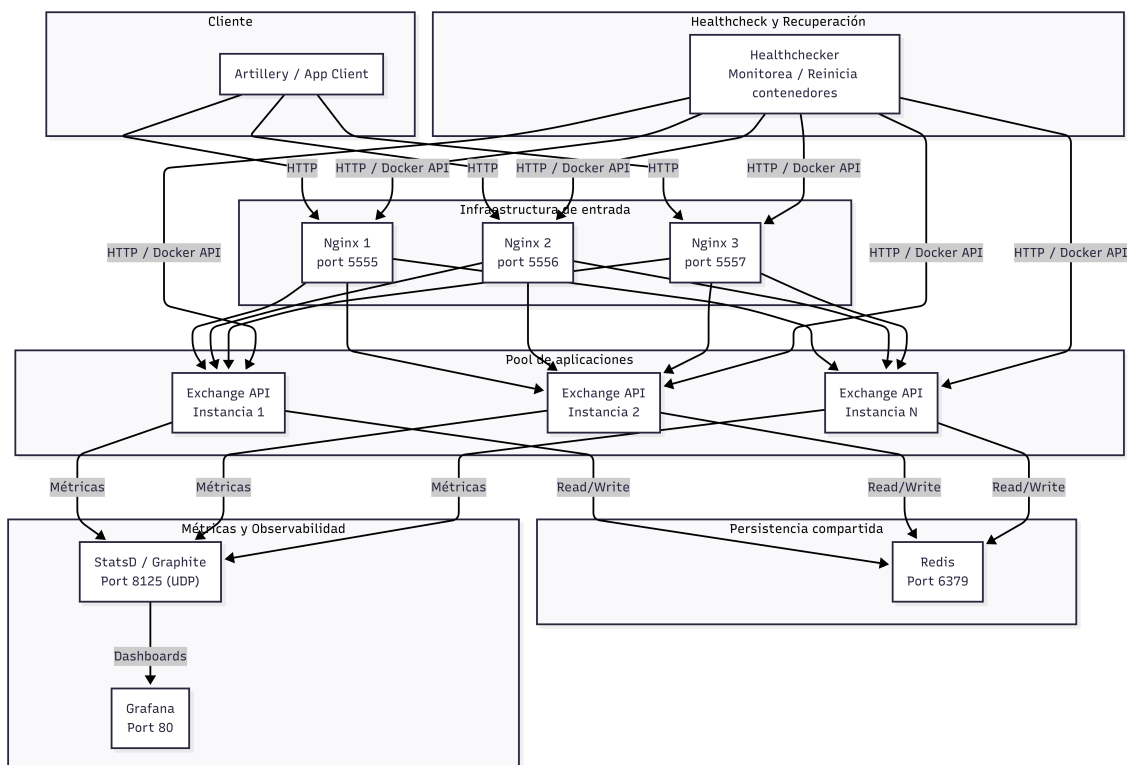


Figura 49: Components & Connector Healthchecker

Cambios de QAs justificados

- **Availability (mejora significativa):** El healthchecker reduce el tiempo durante el cual una caída de api o nginx deja al sistema degradado o sin capacidad de respuesta, automatizando el reinicio de instancias fallidas y evitando depender de intervención manual.
- **Reliability (mejora leve):** El sistema puede distinguir mejor entre una instancia simplemente encendida y una instancia realmente operativa mediante el endpoint de healthcheck.
- **Performance (mejora indirecta):** Los reinicios automáticos y veloces ante fallas minimizan el rendimiento degradado del sistema por disponer de menor cantidad de contenedores.

- **Scalability (mejora indirecta)**: El cambio es necesario para reducir costos de mantenibilidad a medida que crece el sistema horizontalmente.
- **Visibility (mejora)**: Al haber un healthchecker que monitorea constantemente los contenedores y deja logs en el output, se vuelve mucho más fácil trackear el estado del sistema en cuanto a estabilidad.
- **Manageability (mejora significativa)**: La operación mejora al automatizar detección y recuperación de fallas, sin depender de procesos manuales.
- **Simplicity (degrada)**: La arquitectura incorpora un nuevo componente operativo y acceso al socket de Docker.

Mediciones y pruebas

No se incluyen mediciones para este cambio porque el objetivo del healthchecker no es mejorar la performance en condiciones normales, sino reducir el tiempo de recuperación ante fallas. Su impacto principal se observa en escenarios discretos de caída de contenedores que fueron testeados manualmente, donde la evidencia relevante es funcional: detectar la falla, reiniciar la instancia afectada y restaurar el servicio sin intervención manual.

6. Conclusión

A partir del análisis del caso base identificamos un servicio funcionalmente utilizable, pero arquitectónicamente frágil frente a errores de validación, problemas de persistencia y crecimiento de carga. Las pruebas funcionales y de performance permitieron cuantificar esos problemas y usarlos como criterio para evaluar las mejoras introducidas.

Las cuatro tácticas implementadas atacan distintos puntos del sistema. La incorporación de Redis mejora la consistencia del estado y habilita escalabilidad horizontal. El endurecimiento de validaciones reduce respuestas ambiguas y hace al sistema más predecible y testeable. El balanceo con múltiples instancias mejora la disponibilidad y la capacidad de absorber picos de tráfico, a costa de una mayor complejidad operativa mitigada con la incorporación del Healthchecker.

En conjunto, el trabajo muestra que las mejoras arquitectónicas no eliminan todos los costos, pero sí desplazan el sistema hacia un perfil más adecuado para un servicio financiero: más confiable, más observable y más escalable, con trade-offs explícitos y medibles.